Letter

Latent Factorization of Tensors in Hyperbolic Space for Spatiotemporal Traffic Data Imputation

Hao Wu, *Member, IEEE*, Lei Yang, and Zhetao Zhang

Dear Editor,

This letter presents a novel Hyperbolic Latent Factorization of Tensors (HLFT) model for spatiotemporal traffic data imputation. High-quality spatiotemporal traffic data is vital in various intelligent traffic applications. Owing to the malfunctions of sensors or unstable data transmission in complex traffic scenarios, traffic data commonly suffer from severe incompleteness or corruption, making their reliable imputation greatly important for facilitating various subsequent tasks like smart routing. A latent factorization of tensors (LFT) model is frequently utilized for missing traffic data imputation. However, an existing LFT model is commonly built in a Euclidean space. Recent research demonstrates that a hyperbolic space is roomier than a Euclidean space, where a hyperbolic vector can represent richer information than a Euclidean vector does. Motivated by this discovery, this letter innovatively presents the HLFT model with two-fold ideas: 1) defining the model learning objective in a hyperbolic space for high imputation accuracy owing to the high representation ability of the hyperbolic vector, and 2) building a parameter learning scheme according to the hyperbolic space characteristics for efficient model training. Extensive experimental results on four real traffic datasets reveal that the proposed HLFT model obtains significantly computational efficiency and prediction accuracy gains when tasked with the missing spatiotemporal traffic data imputation in comparison with state-of-the-art models.

The Intelligent Transportation System (ITS) stands out as a highly potent remedy for a multitude of traffic issues. High-quality spatiotemporal traffic data is a fundamental element in the ITS. However, owing to various unavoidable factors such as sensor corruption or transmission failure, the challenge of incompleteness in traffic data continues to pose a significant obstacle, complicating traffic analytics. Consequently, accurately estimating missing spatiotemporal traffic data has become a crucial yet complex issue.

Related work: To date, spatiotemporal traffic data imputation has become a focal point for researchers, resulting in the development of diverse models for estimating missing data. Among them, a LFT model has proven to be highly efficient. Consequently, it has found widespread use in traffic data imputation tasks. Given an incomplete traffic data tensor, an LFT-based estimator works by building a low-rank approximation to the target incomplete tensor depending on a set of latent feature (LF) matrices [1]. For example, a transforms-based Bayesian tensor completion method [2], an LFT-based regularized tensor factorization method [3], an enhanced CP completion method [4], a multi-task neural tensor factorization model [5], and an LFT-based customized tensor factorization framework [6].

Although existing LFT-based spatiotemporal traffic data imputation models have been successful, prior research has exclusively focused on addressing the problem within the confines of Euclidean space, where traffic data tensors are represented into Euclidean vectors. However, recent investigations indicate that hyperbolic space offers greater capacity than Euclidean space [7]. Hyperbolic vectors have the ability to encapsulate more diverse and nuanced information compared to Euclidean vectors of equivalent dimensions. For example, Tan *et al.* [8] model the complex relations among users, items, and tags in a unified hyperbolic space for performing recommendation accurately. Li *et al.* [9] design a deep hyperbolic representation learning method that exploits mutual semantic relationships among users/items for collaborative filtering tasks. Wang *et al.* [10] propose a self-guided random walk method for embedding heterogeneous networks into hyperbolic space. Thus, this work aims to design a HLFT model, where the low-rank approximation tensor of a target incomplete traffic data tensor and corresponding LF matrices are learned in hyperbolic space, thereby achieving accurate estimations for missing values.

Problem statement: When performing completion of missing traffic data, a "sensor-time interval-day" third-order incomplete tensor $\mathbf{X}^{|I|\times|J|\times|K|}\in\mathbb{R}$ ($\mathbb{R}$ denotes the real number domain) is taken as a fundamental input data source, as depicted in Fig. 1, where each known element x_{ijk} indicates the speed collected by sensor $i\in I$ on time interval $j\in J$ at day $k\in K$. This study applies Canonical Polyadic tensor factorization framework to perform the LFT on $\mathbf{X}$ [1]. Through this method, $\mathbf{X}$ is decomposed into R rank-one tensors $\mathbf{A}_r$ ($r=1,\ldots,R$), where R is the rank of a resultant approximation $\hat{\mathbf{X}}$ to $\mathbf{X}$. Furthermore, R rank-one tensors $\mathbf{A}_r$ can be obtained by three LF matrices $\mathrm{S}^{|I|\times R}\in\mathbb{R}$, $\mathrm{D}^{|J|\times R}\in\mathbb{R}$, $\mathrm{T}^{|K|\times R}\in\mathbb{R}$, that is to say, a rank-R LFT model decomposes $\mathbf{X}$ into three LF matrices, as shown in Fig. 1.

Note that this study performs the LFT on $\mathbf{X}$ in hyperbolic space. Normally, hyperbolic space is defined as a fully connected Riemannian manifold with constant negative curvature. This constant negative curvature gives hyperbolic space distinctive geometrical properties that sharply contrast with those of Euclidean space, which features constant positive curvature [7]. In particular, there are five isometric models that can be used to describe hyperbolic space, the R-dimensional Poincaré ball model is well-suited for gradient-based optimization, thus, it is adopted in this study:

$$\mathbb{B}^{d,c}=\left\{\boldsymbol{x}\in\mathbb{R}^d:\|\boldsymbol{x}\|^2<\frac{1}{c}\right\} \tag{1}$$

where $-c$ ($c>0$) denotes the negative curvature.

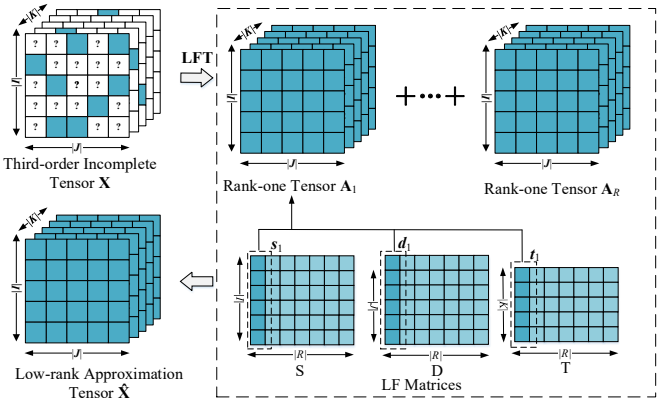


Fig. 1. Rank-one tensors and LF matrices in LFT.

Accordingly, the distance between points $\boldsymbol{a}$ and $\boldsymbol{b}$ on the manifold $\mathbb{B}^{d,c}$ is defined as:

$$d_c(\boldsymbol{a},\boldsymbol{b})=\operatorname{arcosh}\left(1+2\frac{\|\boldsymbol{a}-\boldsymbol{b}\|^2}{(1-\|\boldsymbol{a}\|^2)(1-\|\boldsymbol{b}\|^2)}\right) \tag{2}$$

where $\operatorname{arcosh}(\cdot)$ is the inverse hyperbolic cosine function, $\|\cdot\|$ denotes the Euclidean norm.

In hyperbolic space, in order to acquire the desired LF matrices S, D and T by performing the LFT model, we require LF vectors $\boldsymbol{s}_i$ (the i-th column vector of S), $\boldsymbol{d}_j$ (the j-th column vector of D) and $\boldsymbol{t}_k$ (the k-th column vector of T) to be in the r-dimensional hyperbolic space $\mathbb{R}^r$ with $-c$ as the negative curvature, thus, each element $\hat{x}_{ijk}$ in $\hat{\mathbf{X}}$ can be formulated as:

$$\hat{x}_{ijk}=f(\boldsymbol{s}_i,\boldsymbol{d}_j,\boldsymbol{t}_k,c). \tag{3}$$

The proposed HLFT model: As prior research [7], in hyperbolic space, $\hat{x}_{ijk}$ in (3) can be reformulated as:

Corresponding author: Hao Wu.

Citation: H. Wu, L. Yang, and Z. Zhang, "Latent factorization of tensors in hyperbolic space for spatiotemporal traffic data imputation," *IEEE/CAA J. Autom. Sinica*, vol. 13, no. 8, pp. 1991–1993, Aug. 2026.

H. Wu is with the College of Computer and Information Science, Southwest University, Chongqing 400715, China (e-mail: haowuf@swu.edu.cn).

L. Yang is with the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: d230201043@stu.cqupt.edu.cn).

Z. Zhang is with the College of Arts and Sciences, The University of North Carolina at Chapel Hill, Chapel Hill, NC 27599, USA (e-mail: zhetaozhang719@unc.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/JAS.2024.124911

$$\hat{x}_{ijk} = \delta \cdot (1 - \tanh p_{ijk}) \tag{4}$$

where δ denotes a hyperparameter used to adjust the scale of tensor values, $\tanh(\cdot)$ is the hyperbolic tangent function. p_{ijk} denotes the sum of all distances between any pair of two LF vectors for an element $\hat{x}_{ijk}$, it can be calculated as:

$$p_{ijk} = d_c(\boldsymbol{s}_i, \boldsymbol{d}_j) + d_c(\boldsymbol{s}_i, \boldsymbol{t}_k) + d_c(\boldsymbol{d}_j, \boldsymbol{t}_k). \tag{5}$$

To acquire the desired LF vectors $\boldsymbol{s}_i$, $\boldsymbol{d}_j$ and $\boldsymbol{t}_k$, a learning objective is necessary for measuring the difference between $\mathbf{X}$ and $\hat{\mathbf{X}}$, and L_2 norm is frequently chosen to model such difference. According to prior research [1], an LFT-based model typically follows a data density-oriented modeling principle to enhance computational efficiency, where an objective function is defined on the known data set Λ of an incomplete tensor only. Thus, the objective function is formulated as:

$$\begin{aligned}\varepsilon &= \sum_{x_{ijk}\in\Lambda} \left(x_{ijk} - \hat{x}_{ijk}\right)^2 \\ &= \sum_{x_{ijk}\in\Lambda} \left(x_{ijk} - \delta\left(1 - \tanh\left(d_c\left(\boldsymbol{s}_i, \boldsymbol{d}_j\right) + d_c\left(\boldsymbol{s}_i, \boldsymbol{t}_k\right) + d_c\left(\boldsymbol{d}_j, \boldsymbol{t}_k\right)\right)\right)\right)^2.\end{aligned} \tag{6}$$

The Riemannian stochastic gradient descent (RSGD) algorithm [11] is applied for solving the HLFT model due to its easy implementation and low computational cost. It can consider the instant loss ε_{ijk} on a single entry x_{ijk} from (6):

$$\varepsilon_{ijk} = \left(x_{ijk} - \delta\left(1 - \tanh\left(d_c\left(\boldsymbol{s}_i, \boldsymbol{d}_j\right) + d_c\left(\boldsymbol{s}_i, \boldsymbol{t}_k\right) + d_c\left(\boldsymbol{d}_j, \boldsymbol{t}_k\right)\right)\right)\right)^2. \tag{7}$$

Furthermore, according to RSGD, the LF vectors need to be remained within the Poincaré ball via the following projection:

$$\text{proj}(\theta) = \begin{cases} \theta / \|\theta\| - \tau, & \text{if } \|\theta\| \geq 1 \\ \theta, & \text{otherwise} \end{cases} \tag{8}$$

where $\tau = 10^{-5}$ serves as a small constant to ensure numerical stability. With RSGD, each LF vector at (n+1)-th iteration is updated as follows:

$$\boldsymbol{s}_i^{n+1} \leftarrow \text{proj}\left(\boldsymbol{s}_i^n - \eta_s \cdot \left(1 - \left\|\boldsymbol{s}_i^n\right\|^2\right)^2 \Big/ 4 \cdot \left(\partial\varepsilon\left(\boldsymbol{s}_i^n\right) \Big/ \partial \boldsymbol{s}_i^n\right)\right) \tag{9}$$

$$\boldsymbol{d}_j^{n+1} \leftarrow \text{proj}\left(\boldsymbol{d}_j^n - \eta_d \cdot \left(1 - \left\|\boldsymbol{d}_j^n\right\|^2\right)^2 \Big/ 4 \cdot \left(\partial\varepsilon\left(\boldsymbol{d}_j^n\right) \Big/ \partial \boldsymbol{d}_j^n\right)\right) \tag{10}$$

$$\boldsymbol{t}_k^{n+1} \leftarrow \text{proj}\left(\boldsymbol{t}_k^n - \eta_t \cdot \left(1 - \left\|\boldsymbol{t}_k^n\right\|^2\right)^2 \Big/ 4 \cdot \left(\partial\varepsilon\left(\boldsymbol{t}_k^n\right) \Big/ \partial \boldsymbol{t}_k^n\right)\right). \tag{11}$$

where η_s, η_d, and η_t denote learning rates for $\boldsymbol{s}_i$, $\boldsymbol{d}_j$, and $\boldsymbol{t}_k$. Note that to accommodate the impacts of negative curvature within hyperbolic space, the learning rate needs to be adaptively calculated based on the specific location [7], thus, three learning rates η_s, η_d, and η_t are reformulated as:

$$\eta_s = \eta_0 \cdot e^{-\|\boldsymbol{s}_i\|},\ \eta_d = \eta_0 \cdot e^{-\|\boldsymbol{d}_j\|},\ \eta_t = \eta_0 \cdot e^{-\|\boldsymbol{t}_k\|}. \tag{12}$$

where $\eta_0 = 0.001$ denotes the constant step size used for updating at the initial point. To maintain conciseness, we give the learning rule with $\boldsymbol{s}_i$ only since those for $\boldsymbol{d}_j$ and $\boldsymbol{t}_k$ share similar expressions. Thus, the gradient for $\boldsymbol{s}_i$ in (9) is formulated by:

$$\partial\varepsilon(\boldsymbol{s}_i)/\partial\boldsymbol{s}_i = \Delta_{ijk} \cdot \left(\varphi(\boldsymbol{s}_i, \boldsymbol{d}_j) + \varphi(\boldsymbol{s}_i, \boldsymbol{t}_k)\right) \tag{13}$$

where $\Delta_{ijk} = 2(x_{ijk} - \hat{x}_{ijk}) \cdot \varphi \cdot (1 - \tanh^2 p_{ijk})$, $\varphi(\boldsymbol{s}_i, \boldsymbol{d}_i)$ represents the partial derivate of the Poincaré distance with respect to the first vector $\boldsymbol{s}_i$:

$$\varphi\left(\boldsymbol{s}_i, \boldsymbol{d}_j\right) = \frac{\partial d_c(\boldsymbol{s}_i, \boldsymbol{d}_j)}{\partial \boldsymbol{s}_i} = \frac{4}{\beta\sqrt{\gamma^2 - 1}}\left(\frac{\left\|\boldsymbol{d}_j\right\|^2 - 2\left\langle \boldsymbol{s}_i, \boldsymbol{d}_j\right\rangle + 1}{\alpha^2}\boldsymbol{s}_i - \frac{\boldsymbol{d}_j}{\alpha}\right) \tag{14}$$

where $\alpha = 1 - \|\boldsymbol{s}_i\|^2$, $\beta = 1 - \|\boldsymbol{d}_j\|^2$, $\gamma = 1 + (2\|\boldsymbol{s}_i - \boldsymbol{d}_j\|^2 / \alpha\beta)$. $\rho(\boldsymbol{s}_i, \boldsymbol{d}_j)$ represents the partial derivate of the Poincaré distance with respect to the second vector $\boldsymbol{d}_j$:

$$\rho\left(\boldsymbol{s}_i, \boldsymbol{d}_j\right) = \frac{\partial d_c(\boldsymbol{s}_i, \boldsymbol{d}_j)}{\partial \boldsymbol{d}_j} = \frac{4}{\alpha\sqrt{\gamma^2 - 1}}\left(\frac{\|\boldsymbol{s}_i\|^2 - 2\left\langle \boldsymbol{d}_j, \boldsymbol{s}_i\right\rangle + 1}{\beta^2}\boldsymbol{d}_j - \frac{\boldsymbol{s}_i}{\beta}\right). \tag{15}$$

Therefore, we can achieve the parameters learning scheme for $\boldsymbol{s}_i$, $\boldsymbol{d}_j$, and $\boldsymbol{t}_k$ as follows:

$$\boldsymbol{s}_i^{n+1} \leftarrow \text{proj}\left(\boldsymbol{s}_i^n - \eta_0 \cdot e^{-\|\boldsymbol{s}_i^n\|} \cdot \left(1 - \left\|\boldsymbol{s}_i^n\right\|^2\right)^2 \Big/ 4 \cdot \Delta_{ijk} \cdot \left(\varphi\left(\boldsymbol{s}_i^n, \boldsymbol{d}_j^n\right) + \varphi\left(\boldsymbol{s}_i^n, \boldsymbol{t}_k^n\right)\right)\right) \tag{16}$$

$$\boldsymbol{d}_j^{n+1} \leftarrow \text{proj}\left(\boldsymbol{d}_j^n - \eta_0 \cdot e^{-\|\boldsymbol{d}_j^n\|} \cdot \left(1 - \left\|\boldsymbol{d}_j^n\right\|^2\right)^2 \Big/ 4 \cdot \Delta_{ijk} \cdot \left(\rho\left(\boldsymbol{s}_i^n, \boldsymbol{d}_j^n\right) + \varphi\left(\boldsymbol{d}_j^n, \boldsymbol{t}_k^n\right)\right)\right) \tag{17}$$

$$\boldsymbol{t}_k^{n+1} \leftarrow \text{proj}\left(\boldsymbol{t}_k^n - \eta_0 \cdot e^{-\|\boldsymbol{t}_k^n\|} \cdot \left(1 - \left\|\boldsymbol{t}_k^n\right\|^2\right)^2 \Big/ 4 \cdot \Delta_{ijk} \cdot \left(\rho\left(\boldsymbol{s}_i^n, \boldsymbol{t}_k^n\right) + \rho\left(\boldsymbol{d}_j^n, \boldsymbol{t}_k^n\right)\right)\right). \tag{18}$$

Building upon the aforementioned conclusions, we formulate the algorithm for the HLFT model, outlined in Algorithm 1. According to the Algorithm 1, we see that the computational complexity of the HLFT is $\Theta(n \times |\Lambda| \times R)$, and the storage complexity of the HLFT is $\Theta((|\Lambda| + (|I| + |J| + |K|) \times R)$, they are linear with the known data set $|\Lambda|$.

Algorithm 1 HLFT

Input: $|\Lambda|$, R, φ, η_0

Operation		**Cost**
1.	**Initialize:** $\mathrm{S}^{|I|\times R}$, $\mathrm{D}^{|J|\times R}$, $T^{|K|\times R}$ randomly	$\Theta((|I| + |J| + |K|) \times R)$
2.	**Initialize** $n = 1$, $N =$ *max_iteration_count*	$\Theta(1)$
3.	**while** $n \leq N$ **and not** converge **do**	$\times n$
4.	**for each** x_{ijk} **in** Λ	$\times|\Lambda|$
5.	**Compute** $\hat{x}_{ijk}$ with (4)	$\Theta(R)$
6.	**Update** $\boldsymbol{s}_i$ with (16)	$\Theta(R)$
7.	**Update** $\boldsymbol{d}_j$ with (17)	$\Theta(R)$
8.	**Update** $\boldsymbol{t}_k$ with (18)	$\Theta(R)$
9.	**end for**	--
10.	$n = n$+1	$\Theta(1)$
11.	**end while**	--

Output: S, D, T

Experiments: For evaluating the performance of HLFT, Berlin speed data[1], Guangzhou speed data[2], Seattle freeway speed data[3], and New York speed data[4] are used in our experiments. Moreover, all the speed data are normalized to eliminate biased results caused by magnitude differences. Each data set is randomly divided into a training set K, a validation set Ψ, and a testing set Ω, following a ratio of 7:1:2. The root mean squared error (RMSE) and mean absolute error (MAE) are used as the indicator to evaluate the accuracy of involved models [12], i.e.,

$$\text{RMSE} = \sqrt{\sum_{x_{ijk}\in\Omega}\left(x_{ijk} - \hat{x}_{ijk}\right)^2 \Big/ |\Omega|},\ \text{MAE} = \sum_{x_{ijk}\in\Omega}\left|x_{ijk} - \hat{x}_{ijk}\right| \Big/ |\Omega|.$$

Note that low RMSE and MAE indicate high prediction accuracy of a tested model for missing spatiotemporal traffic data. Additionally, the time efficiency of a tested model, as reflected by the training time recorded for each dataset, is also taken into consideration.

We compare the proposed HLFT model (M1) with five state-of-the-art models: GC_MC (M2) [13], LightGCN$_{\text{r-AdjNorm}}$ (M3) [12], SGL-ED (M4) [14], DGCN-HN (M5) [15] and GTN (M6) [16]. Note that M1−6 are implemented in Python, where M1 is conducted on a Tablet with a 2.2 GHz Intel Core CPU and 16 GB RAM and M2−6 are conducted on a GPU platform with 32 GB RAM and one NVIDIA GeForce RTX 2080 TI GPU card. Note that the negative curvature is set at -1(c=1). The latent feature space dimension is set as 20 uniformly [1]. The value of δ is determined as the maximum of the target incomplete third-order tensor values.

Fig. 2 illustrates the performance comparison of M1−6 in terms of prediction accuracy and total time cost on D1−4. Based on these

[1] https://daten.berlin.de

[2] https://zenodo.org/record/1205229

[3] https://github.com/zhiyongc/Seattle-Loop-Data

[4] https://opendata.cityofnewyork.us/

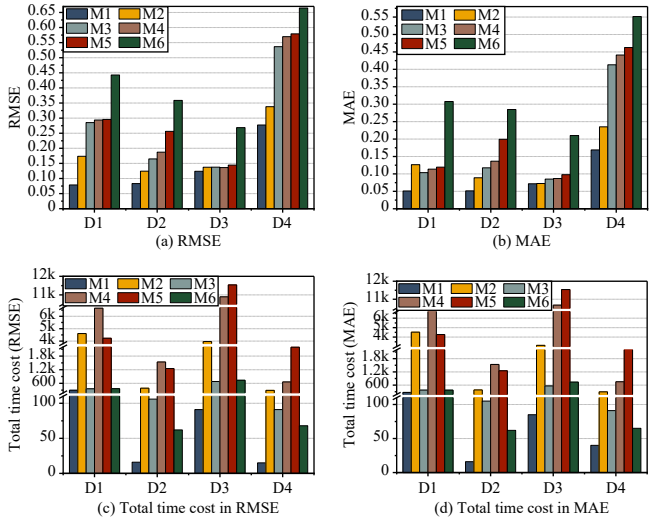


Fig. 2. Performance comparison of M1−6 on D1−4.

results, the following observations are made:

a) **M1, i.e., the proposed HLFT model, outperforms its peers in terms of prediction accuracy for missing data of an incomplete traffic data tensor**. As illustrated in Fig. 2(a), on D3, M1's RMSE is 0.1239, which is approximately 9.76% lower than M2's 0.1373, 9.96% lower than M3's 0.1376, 9.10% lower than M4's 0.1363, 14.20% lower than M5's 0.1444, and 53.84% lower than M6's 0.2684. Considering MAE, as shown in Fig.2(b), the outputs by M1-6 are 0.0716, 0.0725, 0.0851, 0.0867, 0.0978, and 0.2101, respectively. Therefore, M1 demonstrates significantly lower MAE compared to its peers. Similar results are observed on D1 and D3−4, as shown in Figs. 2 (a) and 2 (b). Note that M2−6 are built on Euclidean space, thus, M1's high prediction accuracy mainly relies on the fact that hyperbolic space offers greater representation capacity than Euclidean space in the same dimensions.

b) **HLFT obtains high computational efficiency on incomplete spatiotemporal traffic data tensor imputation.** As depicted in Fig. 2(c), M1 takes 91 seconds to converge in RMSE on D3, which is mere 2.29% of 3966 seconds by M2, 13.36% of 681 seconds by M3, 0.84% of 10895 seconds by M4, 0.79% of 11537 seconds by M5, and 12.40% of 734 seconds by M6. Similar outcomes are also observed on D1 and D3−4 as shown in Fig. 2(c). In terms of MAE, on D1−4, M1 respectively takes 267 seconds, 16 seconds, 85 seconds and 40 seconds to converge, which are considerably less than the convergence times of M2−6 as illustrated in Fig. 2(d). In particular, the high efficiency of HLFT is mainly because the computational complexity of HLFT is $\Theta(n\times|\Lambda|\times R)$ as $|\Lambda|$ denotes the known element number of an incomplete traffic data tensor, so there is no need to consider an incomplete tensor's unknown elements when training the HLFT, which makes HLFT have inherent high computational efficiency.

c) **The performance improvements of M1 in prediction accuracy and computational efficiency are notable.** To gain deeper insights into these results, we conducted Wilcoxon signed-ranks tests with three key indicators: R+, R−, and *p*-value. A higher R+ value signifies better model performance, while the *p*-value indicates the level of significance. Table 1 summarizes the Wilcoxon signed-ranks test results with the significance level of 0.1 corresponding to the findings in Fig. 2, focusing on RMSE/MAE and total time cost across different datasets. This analysis comprises 8 cases for 6 tested models. As demonstrated in Table 1, M1 (HLFT model) achieves significant accuracy and efficiency gains over its counterparts, evidenced by consistently high R+ values and significant *p*-values. Notably, M1 outperforms all compared models across all test cases, as indicated by 10 zeroes for the R− value out of 10 comparisons in total.

Table 1. Wilcoxon Signed-Ranks Test Results

Comparison	Figs.2(a)−(b)			Figs.2(c)−(d)		
	R+	R−	*p*-value	R+	R−	*p*-value
M1 vs. M2	36	0	0.0039	36	0	0.0039
M1 vs. M3	36	0	0.0039	36	0	0.0039
M1 vs. M4	36	0	0.0039	36	0	0.0039
M1 vs. M5	36	0	0.0039	36	0	0.0039
M1 vs. M6	36	0	0.0039	36	0	0.0039

Conclusions: Aiming at establishing a precise spatiotemporal traffic data imputation model, this letter proposes a HLFT model that defines the learning objective in hyperbolic space and designs a parameter learning scheme applicable to the property of hyperbolic space. The empirical results show that the proposed HLFT model significantly outperforms its peers in both prediction accuracy and computational efficiency. Moving forward, we intend to incorporate regularization schemes, parameter adaptive methods, or advanced optimization methods to further enhance the model's performance.

Acknowledgments: This work was supported by the National Natural Science Foundation of China (62302402), the Science and Technology Research Program of Chongqing Municipal Education Commission (KJZD-K202400209).

References

[1] H. Wu, X. Wu, and X. Luo, *Dynamic network representation based on latent factorization of tensors*. Singapore: Springer Singapore, 2023.

[2] Z. Yang, L. T. Yang, L. Yi, X. Deng, C. Zhu, and Y. Ruan, "Transforms-based Bayesian tensor completion method for network traffic measurement data recovery," *IEEE Trans. on Network Science and Engineering*, vol. 11, no. 3, pp. 2497−2509, 2024.

[3] A. Baggag, S. Abbar, A. Sharma, T. Zanouda, A. Al-Homaid, A. Mohan, and J. Srivastava, "Learning spatiotemporal latent factors of traffic via regularized tensor factorization: imputing missing values and forecasting," *IEEE Trans. Knowledge and Data Engineering*, vol. 33, no. 6, pp. 2573−2587, 2021.

[4] A. Ben Said and A. Erradi, "Spatiotemporal tensor completion for improved urban traffic imputation," *IEEE Trans. Intelligent Transportation Systems*, vol. 23, no. 7, pp. 6836−6849, 2022.

[5] Y. Zhu, J. Wang, J. Wang, and Z. He, "Multitask neural tensor factorization for road traffic speed-volume correlation pattern learning and joint imputation," *IEEE Trans. Intelligent Transportation Systems*, vol. 23, no. 12, pp. 24550−24560, 2022.

[6] J. Xing, R. Liu, K. Anish, and Z. Liu, "A customized data fusion tensor approach for interval-wise missing network volume imputation," *IEEE Trans. Intelligent Transportation Systems*, vol. 24, no. 11, pp. 12107−12122, 2023.

[7] B. Hui and W. Ku. "Low-rank nonnegative tensor decomposition in hyperbolic space," in *Proc. of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Washington, DC, USA, pp. 646−654, 2022.

[8] Y. Tan, C. Yang, X. Wei, C. Chen, L. Li, and X. Zheng, "Enhancing recommendation with automated tag taxonomy construction in hyperbolic space," in *Proc. of the IEEE 38th International Conference on Data Engineering (ICDE)*, Kuala Lumpur, Malaysia, 2022, pp. 1180−1192.

[9] A. Li, B. Yang, H. Huo, H. Chen, G. Xu, and Z. Wang, "Hyperbolic neural collaborative recommender," *IEEE Trans. on Knowledge and Data Engineering*, vol. 35, no. 9, pp. 9114−9127, 2023.

[10] L. Wang, C. Gao, C. Huang, R. Liu, W. Ma, and S. Vosoughi, "Embedding heterogeneous networks into Hyperbolic space without Meta-path", *Proc. of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 11, pp. 10147−10155, 2021.

[11] M. Nickel and D. Kiela, "Poincaré embeddings for learning hierarchical representations," in *Proc. of the 31st conference on Neural Information Processing Systems*, Long Beach, CA, USA, 2017, pp. 6341−6350.

[12] M. Zhao, L. Wu, Y. Liang, L. Chen, J. Zhang, Q. Deng, K. Wang, X. Shen, T. Lv, and R. Wu, "Investigating accuracy-novelty performance for graph-based collaborative filtering," in *Proc. of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, Madrid, Spain, 2022, pp. 50−59.

[13] R. Berg, T. N. Kipf, and M. Welling, "Graph convolutional matrix completion," in *Proc. of the 24th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, London, UK, 2018.

[14] J. Wu, X. Wang, F. Feng, X. He, L. Chen, J. Lian, and X. Xie, "Self-supervised graph learning for recommendation," in *Proc. of the 44th international ACM SIGIR conference on research and development in information retrieval*, Virtual Event, Canada, 2021, pp. 726−735.

[15] W. Guo, Y. Yang, Y. Hu, C. Wang, H. Guo, Y. Zhang, R. Tang, W. Zhang, and X. He, "Deep graph convolutional networks with hybrid normalization for accurate and diverse recommendation," in *Proc. of 3rd Workshop on Deep Learning Practice for High-Dimensional Sparse Data with KDD*, Singapore, 2021.

[16] W. Fan, X. Liu, W. Jin, X. Zhao, J. Tang, and Q. Li, "Graph trend filtering networks for recommendation," in *Proc. of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, Madrid, 2022, pp. 112−121.